



**UNIVERSIDADE ESTADUAL DA PARAÍBA
CAMPUS I – EDVALDO DE SOUZA DO Ó
CENTRO DE CIÊNCIAS E TECNOLOGIA
CURSO DE BACHAREL EM CIÊNCIA DA COMPUTAÇÃO**

**Gabriela Almeida de Andrade
Georis Samuel Martins Gouveia
Horlan Silva de Lacerda
Suelle Ferreira Maciel**

**ESTUDO COMPARATIVO DE TÉCNICAS DE PROJETO DE ALGORITMO
APLICADO AO PROBLEMA COMPUTACIONAL N-QUEENS**

**CAMPINA GRANDE, PB
2026**

GABRIELA ALMEIDA DE ANDRADE
GEORIS SAMUEL MARTINS GOUVEIA
HORLAN SILVA DE LACERDA
SUELLE FERREIRA MACIEL

**ESTUDO COMPARATIVO DE TÉCNICAS DE PROJETO DE ALGORITMO
APLICADO AO PROBLEMA COMPUTACIONAL N-QUEENS**

Relatório apresentado à disciplina
Técnicas de Análise de Algoritmo do Curso
de Bacharelado em Ciência da
Computação da Universidade Estadual da
Paraíba (UEPB), como requisito parcial
para obtenção de nota na Unidade
Acadêmica

Orientador(a): Prof. Dr. Fábio Luiz Leite Júnior

**CAMPINA GRANDE, PB
2026**

RESUMO

Este relatório apresenta um estudo comparativo entre três abordagens algorítmicas aplicadas ao problema clássico das N-Rainhas (N-Queens): Backtracking, Backtracking com Bit-masking (variação otimizada do Backtracking tradicional) e Branch and Bound, implementadas na linguagem de programação JAVA. O trabalho avalia desempenho (tempo de execução e consumo de memória), exploração do espaço de busca (estados explorados, podas realizadas), qualidade da solução e escalabilidade, comparando ainda a complexidade teórica com o comportamento observado experimentalmente. A pesquisa realizada visa demonstrar como a escolha de um algoritmo pode influenciar diretamente na resolução de um problema computacional, avaliando desempenho, a exploração do espaço de busca e a qualidade das soluções obtidas em cada abordagem.

Palavras-chave: N-Rainhas; Backtracking; Backtracking com Bit-masking; Branch and Bound;

ABSTRACT

This report presents a comparative study of three algorithmic approaches applied to the classic N-Queens problem: Backtracking, Backtracking with Bit-masking (an optimized variation of traditional Backtracking), and Branch and Bound, implemented in the Java programming language. The study evaluates performance (execution time and memory usage), search space exploration (explored states, pruning performed), solution quality, and scalability, while also comparing theoretical complexity with experimentally observed behavior. The research aims to demonstrate how the choice of algorithm can directly influence the resolution of a computational problem, by evaluating performance, search space exploration, and the quality of the solutions obtained in each approach.

Keywords: N-Queens; Backtracking; Backtracking with Bit-masking; Branch and Bound;

LISTA DE ILUSTRAÇÕES

Figura 1 - Diagonais da matriz quadrada 8 x 8.....	11
Figura 2 - Backtracking Aplicado ao Problema das N-Queens.....	13
Figura 3 - Tempo médio de execução em escala logarítmica em função do tamanho da entrada (N) por máquina	29
Figura 4 - Comparativo de tempo de execução entre os algoritmos para o caso crítico (N = 14) em diferentes ambientes de hardware	29
Figura 5 - Consumo médio de memória dos algoritmos em função do tamanho da entrada (N)	30
Figura 6 - Comparativo de consumo de memória dos algoritmos entre os ambientes de execução	30
Figura 7 - Número de chamadas recursivas e quantidade de podas por abordagem no problema das N-Rainhas.....	31
Figura 8 - Tempo de execução no ponto de ruptura por abordagem sob o limiar de 1 minuto em diferentes ambientes de hardware.....	33
Figura 9 - Dimensão máxima de entrada (N) processada por abordagem até o esgotamento do limiar temporal	34
Figura 10 - Comportamento do espaço de busca (chamadas recursivas e podas) nos cenários limites de processamento (N = 16, 17, 18)	34

LISTA DE TABELAS

Tabela 1 - Complexidade Assintótica de Problemas que Implementam o Algoritmo do Backtracking.....	13
Tabela 2 - Especificação de Hardware, Software e Sistemas Operacionais Utilizados nos Testes.....	16
Tabela 3 - Organização Modular e Arquitetura do Software Desenvolvido	16
Tabela 4 - Desempenho dos Algoritmos Backtracking, Branch and Bound e Bitmask no problema das N-Rainhas (Máquina 1).....	26
Tabela 5 - Desempenho dos Algoritmos Backtracking, Branch and Bound e Bitmask no problema das N-Rainhas (Máquina 2).....	26
Tabela 6 - Desempenho dos Algoritmos Backtracking, Branch and Bound e Bitmask no problema das N-Rainhas (Máquina 3).....	27
Tabela 7 - Desempenho dos Algoritmos Backtracking, Branch and Bound e Bitmask no problema das N-Rainhas (Máquina 4).....	28
Tabela 8 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 1).....	31
Tabela 9 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 2).....	32
Tabela 10 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 3).....	32
Tabela 11 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 4).....	32

SUMÁRIO

1. INTRODUÇÃO	8
1.1. Problematização e justificativa	9
1.2. Objetivos	9
1.2.1. Objetivo geral	9
1.2.2. Objetivos específicos	9
2. REFERENCIAL TEÓRICO	11
2.1. Problema das N-Rainhas (N-Queens)	11
2.2. Backtracking	12
2.2.1. Backtracking com Bit-masking	14
2.3. Branch and Bound	14
3. METODOLOGIA	16
3.1. Ambiente Computacional e Linguagem Utilizada	16
3.2. Estruturas de Dados e Modelagem do Tabuleiro	17
3.3. Análise de Corretude e Desempenho por algoritmo	18
3.3.1. Backtracking	18
3.3.1.1. Análise de Corretude	18
3.3.1.2. Análise de Desempenho	20
3.3.2. Branch and Bound	20
3.3.2.1. Análise de Corretude	20
3.3.2.2. Análise de Desempenho	22
3.3.3. Backtracking com bit-masking	22
3.3.3.1. Análise de Corretude	22
3.3.3.2. Análise de Desempenho	23
3.4. Critérios Experimentais e Coleta de Métricas	24
4. RESULTADOS	26
4.1. Benchmark de Desempenho entre Algoritmos	26
4.2. Teste de Viabilidade (Ponto de Ruptura)	31
5. DISCUSSÃO	36
6. CONSIDERAÇÕES FINAIS	39
7. DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA (IAGen)	40

1. INTRODUÇÃO

Entende-se por problema computacional aquele cuja resolução se dá por meio de computadores, sendo caracterizado por uma entrada – que reúne parâmetros e dados relevantes ao problema – e por uma saída, que corresponde a resolução esperada do problema. Dessa forma, a atribuição de valores específicos a essa entrada é denominada instância, enquanto que o resultado obtido para a respectiva instância é chamada de solução (LINTZMAYER; MOTA, 2023 apud ANDRADE et al., 2024).

Todo problema computacional demanda uma forma de resolução e essa resolução se dá, segundo Lintzmayer e Mota (2023 apud ANDRADE et al., 2024), por meio de um algoritmo. Este pode ser definido, de forma sucinta, como uma sequência finita e não ambígua de passos que, a partir de um conjunto de dados de entrada, produz uma saída correspondente. Diz-se, ainda conforme os autores, que um algoritmo é correto quando devolve uma solução válida para qualquer instancia do problema ao qual se destina.

O problema das N-Rainhas ilustra bem essa relação entre problema e solução algorítmica. O problema surgiu, inicialmente, com apenas 8 rainhas, proposto pela primeira vez por Max Bezzel em 1848, o qual tinha como desafio preencher um tabuleiro de xadrez com 8 rainhas. Neste tabuleiro, nenhuma delas poderia conflitar com a outra, levando em consideração os movimentos de ataque da peça. No entanto, em 1850, o problema foi generalizado por Nauck, que propôs um valor arbitrário n para a quantidade de rainhas dispostas no tabuleiro e criou uma nova variação do problema: a completude das n -rainhas (SOUZA, 2019).

Ainda segundo Souza (2019), já em 1965, o problema passou a despertar o interesse não somente dos matemáticos e enxadristas, mas também dos cientistas da computação. Estes começaram a usá-lo como ferramenta de teste para novas técnicas de programação, como a programação Backtracking e Branch and Bound. No entanto, o autor enfatiza que o problema das n -rainhas ainda não possui uma solução geral eficiente, apesar de possuir inúmeras aplicações práticas possíveis.

Portanto, por mais que o problema das n -rainhas não apresente uma solução geral eficaz, é importante a análise das soluções apresentadas – Backtracking, Backtracking com Bit-masking e Branch and Bound – para entender questões de eficiência algorítmica. Com isso, o estudo tem como objetivo geral comparar os

algoritmos, focando em eficiência computacional, qualidade da solução, complexidade e comportamento, contribuindo assim para o conhecimento prático na disciplina de Técnicas de Análise de Algoritmo.

1.1. Problematização e justificativa

A eficiência e o desempenho de algoritmos estão ligados, principalmente, à escolha do paradigma utilizado para resolver um determinado problema. O problema das n -rainhas, apesar de clássico, ainda é amplamente utilizado como parâmetro para testar e comparar diferentes técnicas de programação, dado o crescimento exponencial de seu espaço de busca. Tendo isso em vista, essa pesquisa busca entender na prática o impacto da complexidade teórica de cada abordagem sobre o desempenho real dos algoritmos de Backtracking, Backtracking com Bit-masking e Branch and Bound aplicados ao problema.

Diante deste cenário, esse relatório se justifica, além do viés puramente acadêmico, pela necessidade de conciliar a análise teórica dos algoritmos com sua performance empírica, oferecendo uma perspectiva prática sobre como diferentes estratégias de projeto de algoritmos se comportam diante de um mesmo problema.

1.2. Objetivos

1.2.1. Objetivo geral

Este relatório tem como objetivo geral investigar diferentes estratégias algorítmicas – Backtracking, Backtracking com Bit-masking e Branch and Bound – aplicadas à resolução do problema das n -rainhas, a fim de compreender como cada paradigma se comporta diante desse problema.

1.2.2. Objetivos específicos

- Implementar as estratégias algorítmicas – Backtracking, Backtracking com Bit-masking e Branch and Bound – na linguagem de programação Java.
- Projetar experimentos para avaliação dos algoritmos, utilizando instâncias de diferentes tamanhos.

- Comparar quantitativamente e qualitativamente as soluções obtidas.
- Mensurar o tempo de execução e o consumo de memória de cada algoritmo implementado.
- Analisar a exploração do espaço de busca de cada algoritmo, incluindo número de estados explorados e podas realizadas.
- Produzir uma análise crítica baseada nos resultados experimentais obtidos.

2. REFERENCIAL TEÓRICO

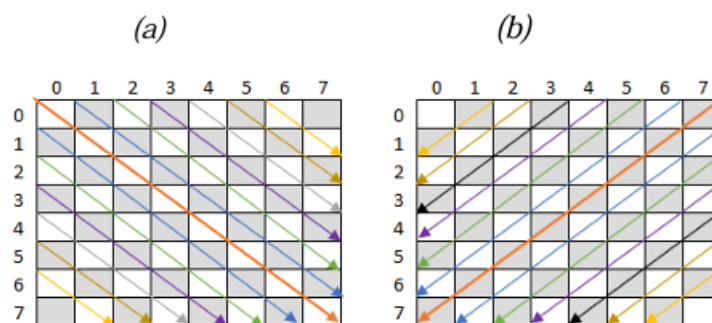
Este capítulo apresenta os conceitos teóricos que fundamentam o desenvolvimento deste trabalho, com o propósito de situar o leitor a respeito das noções necessárias para compreensão da análise de algoritmos abordada nas seções seguintes.

2.1. Problema das N-Rainhas (N-Queens)

Conforme apresentado na introdução, o problema das n-rainhas é estudado há mais de um século e consiste, segundo Oliveira (2024), em posicionar rainhas em um tabuleiro de xadrez sob determinadas restrições. Em termos gerais, essas restrições existem para que as rainhas não se ataquem mutuamente, ou seja, levando em consideração a matriz relativa do tabuleiro constituído por linhas (na horizontal) e colunas (na vertical), pode-se concluir que não podem existir duas rainhas ou mais em uma mesma linha e/ou em uma mesma coluna. Dessa forma, considerando o tabuleiro como uma matriz, a restrição de linha e coluna já reduz consideravelmente o espaço de possibilidades: levando em conta a particularização do problema das n-rainhas para o caso de 8-rainhas, por exemplo, a quantidade de arranjos possíveis é dada por $8! = 40.320$.

Ainda segundo Oliveira (2024), dispondo as rainhas em linhas e colunas, ainda resta analisar se elas não se atacam também nas diagonais. Considerando que a matriz do problema é sempre quadrada, em um tabuleiro 8 x 8, por exemplo, as diagonais seriam exibidas conforme na figura 1 abaixo.

Figura 1 - Diagonais da matriz quadrada 8 x 8



Fonte: OLIVEIRA (2024).

Conforme a figura, são 26 diagonais no caso ilustrado, na quais, consideradas individualmente, não podem ser colocadas duas ou mais rainhas. Sendo assim, são 13 diagonais no mesmo sentido da diagonal principal e 13 no mesmo sentido da diagonal secundária, o que reduz mais ainda mais o espaço de busca (OLIVEIRA, 2024). Ainda assim, por mais que essas restrições reduzam o espaço de busca, o crescimento do problema em função de n torna a força bruta inviável, o que justifica o estudo de estratégias algorítmicas mais eficientes, como é o caso do Backtracking, Backtracking Bit-masking e Branch and Bound.

2.2. Backtracking

O algoritmo do backtracking adota a premissa de que é possível determinar que algumas escolhas iniciais não podem levar a uma solução. Sendo assim, a busca por retrocesso funciona de forma contínua, de modo que o algoritmo sempre busca estender uma solução parcial para um problema. No entanto, se a solução parcial atual não pode se estender para uma solução completa, o algoritmo retrocede para a solução parcial anterior e tenta estendê-la novamente de uma maneira que não foi tentada anteriormente – comportamento que caracteriza a poda de ramos inválidos da árvore de busca, de forma similar a uma árvore de decisão. Esse processo continua até que uma solução seja encontrada ou que não exista solução (KARP; ZHANG, 1993).

Ainda segundo os autores, a premissa do algoritmo se assemelha ao processo de encontrar uma saída de um labirinto: começando na entrada, o caminho é estendido continuamente até que uma saída seja alcançada; ao se deparar com um beco sem saída, retrocede-se um passo e tenta-se estender o caminho em outra direção.

Esse mesmo processo – que pode ser descrito, segundo Massuia (2025), em três etapas de escolha, exploração e retrocesso – é o que implica no custo assintótico $O(b^d)$ do algoritmo, em que b é o fator de ramificação e d é a profundidade da solução. No entanto, essa complexidade assintótica pode variar dependendo do problema ao qual o algoritmo está sendo submetido, o que pode ser visto na Tabela 1 a seguir.

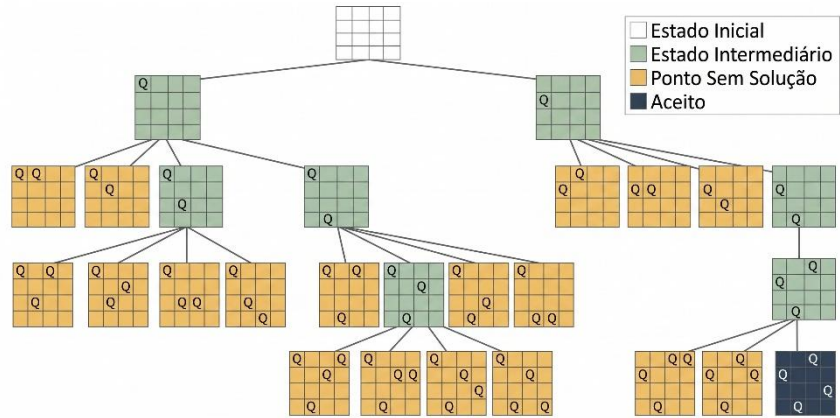
Tabela 1 - Complexidade Assintótica de Problemas que Implementam o Algoritmo do Backtracking

Problema	Pior Caso	Melhor Caso	Caso Médio
Permutação	$O(n!)$	$O(1)$	$O(n!)$
Combinação	$O(2^n)$	$O(1)$	$O(2^n)$
Subconjunto	$O(2^n)$	$O(1)$	$O(2^n)$
Problema das N-Queens	$O(n!)$	$O(1)$	$O(n!)$
Problema do Sudoku	$O(9^n)$	$O(1)$	$O(9^n)$

Fonte: Adaptado de Massuia (2025).

Levando em consideração as complexidades temporais dispostas na tabela e o problema do estudo, evidencia-se que o custo assintótico do problema das n-queens é $O(n!)$ no pior caso. Segundo Rai (2024), o algoritmo, ao ser implementado para esse problema pode, no pior caso, explorar todas disposições possíveis do tabuleiro: para cada linha, tenta-se posicionar uma rainha em todas as colunas, verificando sua segurança a cada tentativa. Na mesma linha, GeeksforGeeks (2026) descreve que o algoritmo avança para a próxima linha ao encontrar uma posição segura, e retrocede para tentar outra posição quando nenhuma coluna da linha atual está livre de conflitos, conforme ilustrado na Figura 2. Dessa forma, a complexidade de tempo pode ser aproximada por $O(n!)$, justamente por existirem $n!$ permutações possíveis de rainhas no tabuleiro.

Figura 2 - Backtracking Aplicado ao Problema das N-Queens



Fonte: Adaptado de GeeksforGeeks (2026).

Além da complexidade temporal, destaca-se também a complexidade espacial, a qual, segundo GeeksforGeeks (2026), é de $O(n^2)$ para o algoritmo Backtracking aplicado ao problema das n -rainhas. Esse comportamento é evidenciado pelo uso de um tabuleiro $n \times n$ e uma pilha de recursão para retroceder.

2.2.1. Backtracking com Bit-masking

Dentro da área da tecnologia, o fator desempenho é crucial quando se trata da resolução de problemas, o que evidencia as diversas variações de algoritmos comuns. A variação do Backtracking com Bit-masking evidencia esse comportamento, tratando-se de uma otimização da versão tradicional com foco em tabuleiros pequenos, conforme evidenciado por GeeksforGeeks (2026). O autor destaca que a ideia central é usar três variáveis para rastrear linhas e diagonais ocupadas, sendo que, a cada passo, são usadas operações de bits para encontrar rapidamente posições seguras. Desconsiderando o fator dos bits, o comportamento do algoritmo é similar ao tradicional: coloca-se uma rainha no tabuleiro, atualizam-se as variáveis e avança-se; se nenhuma solução funcionar, retrocede-se e são tentadas outras opções até encontrar todas as soluções.

E é justamente esse comportamento similar à versão tradicional que faz com que o algoritmo assuma a mesma complexidade temporal de $O(n!)$ no pior caso, evidenciando as $n!$ permutações possíveis para as rainhas. No entanto, a otimização do algoritmo se destaca na complexidade espacial, na qual o algoritmo dispõe de uma complexidade $O(n)$ (GEEKSFORGEEKS, 2026). Isso ocorre porque, em vez de armazenar o tabuleiro completo como uma matriz $n \times n$, a técnica de bit-masking utiliza apenas três variáveis para representar as colunas e diagonais ocupadas, conforme disposto no primeiro parágrafo, reduzindo o espaço adicional necessário de uma ordem quadrática para uma ordem linear em relação a n .

2.3. Branch and Bound

O Branch and Bound é um método de busca em árvore que se fundamenta na programação linear para explorar o espaço de busca: em cada passo, as variáveis inteiras são relaxadas e o subproblema resultante é resolvido por um método de programação linear; se a solução desse subproblema já apresentar todas as variáveis

inteiras, os descendentes daquele ramo são implicitamente enumerados, o que constitui um dos critérios de poda da árvore (SOUZA, 2009). Estruturalmente, essa árvore pode ser descrita como uma árvore de subproblemas, na qual o problema original ocupa a raiz, os filhos de cada subproblema são obtidos por ramificação, e cada folha corresponde a um subproblema resolvido diretamente — sendo o objetivo da busca encontrar a folha de custo mínimo (KARP; ZHANG, 1993).

Aplicado ao problema das n -rainhas, GeeksforGeeks (2025) configura a abordagem como eficiente, dado a redução o número de posições que precisam ser removidas para verificação. Sendo assim, em vez de explorar todas as posições possíveis para as rainhas, o Branch and Bound elimina muitas possibilidades ao reconhecer que alocar uma rainha em uma determinada posição exclui automaticamente várias outras posições da análise, o que proporciona uma maneira mais rápida e eficiente de resolver o problema na prática.

Ainda assim, por mais que o Branch and Bound assuma uma posição superior de eficiência em relação ao Backtracking, a complexidade de tempo do algoritmo no pior caso é também $O(n!)$ para o problema das n -rainhas, onde n representa o número de linhas e colunas no tabuleiro quadrado. Isso ocorre porque, para cada coluna, o algoritmo tenta posicionar uma rainha em cada linha e, em seguida, recursivamente, tenta posicionar as rainhas nas colunas restantes. Logo, o número de combinações possíveis de posicionamento de rainhas no tabuleiro é $n!$, assim como na versão com o backtracking, visto que pode haver apenas uma rainha em cada linha e cada coluna (GEEKSFORGEEKS, 2025).

Nessa mesma linha, o autor ainda pontua que a complexidade espacial do Branch and Bound é $O(n^2)$, onde n representa o número de linhas e colunas no tabuleiro. Esse custo se deve ao fato de o uso de uma matriz 2D para representar o tabuleiro, que ocupa n^2 de espaço.

3. METODOLOGIA

3.1. Ambiente Computacional e Linguagem Utilizada

Os testes dispostos neste relatório foram realizados com o mesmo conjunto de dados, porém em diferentes máquinas, com configurações de hardware, software e sistemas operacionais distintos. A especificação dos equipamentos utilizados nesta pesquisa é detalhada a seguir:

Tabela 2 - Especificação de Hardware, Software e Sistemas Operacionais Utilizados nos Testes

	Máquina 1	Máquina 2	Máquina 3	Máquina 4
RAM	8GB	20GB	8GB	16GB
Processador	Core I5-8265U	Core I5-1235U	Core I7-1255U	Core I5-1335U
OS	Windows 11	Linux Debian 13 KDE	Windows 11	Windows 11
ROM	238GB SSD	512GB SSD	512GB SSD	512GB SSD

Fonte: Elaborado pelos autores (2026).

Juntamente aos dispositivos de hardware descritos, utilizou-se a linguagem de programação Java como ferramenta primária de desenvolvimento, servindo de base tanto para a implementação da lógica dos algoritmos quanto para a realização dos testes de complexidade de tempo. A escolha pela linguagem se justifica tanto pela presença da orientação objetos, que facilita a modularização do código, quanto pela disponibilidade de bibliotecas nativas para a medição de tempo e memória.

Para assegurar a manutenibilidade e a clara separação de responsabilidades, o sistema foi organizado em pacotes modulares, cuja estrutura e respectivos papéis estão sintetizados na Tabela 3.

Tabela 3 - Organização Modular e Arquitetura do Software Desenvolvido

Módulo/Pacote	Artefatos e Classes	Responsabilidade Técnica
com.nqueens	Main	Ponto de entrada do sistema; gerencia o menu iterativo de opções no terminal, orquestra execuções individuais, testes de viabilidade e benchmarks.

com.nqueens. algorithms	Backtracking, BranchAndBound, Bitmask	Implementação das três abordagens exatas para a resolução do problema das N-Rainhas, incluindo rastreamento de nós, podas e soluções.
com.nqueens.core	NQueensAlgorithm (Interface), Metrics	Define o contrato comum para padronização dos algoritmos e encapsula a estrutura de coleta de métricas computacionais (tempo, memória, podas e recursões).
com.nqueens. benchmark	BenchmarkRunner, CsvExporter	Execução automatizada de testes em múltiplas rodadas, cálculo de médias estatísticas e persistência dos dados estruturados em formato .csv.
com.nqueens.ui	Board	Componente de interface responsável por renderizar visualmente a primeira solução válida encontrada na grade do tabuleiro.

Fonte: Elaborado pelos autores (2026).

3.2. Estruturas de Dados e Modelagem do Tabuleiro

A representação do tabuleiro do problema das N-Queens e a gestão de restrições variam de acordo com o paradigma de cada algoritmo implementado, conforme descrito:

- **Backtracking Clássico:** modela o estado por meio de um vetor unidimensional de inteiros `board[N]`, onde o índice indica a linha e o conteúdo define a coluna. A verificação de restrições não emprega memória adicional, sendo computada iterativamente em tempo $O(n)$ no pior caso.
- **Branch and Bound:** mantém o vetor de posições `board[N]` e introduz três vetores booleanos como tabelas de controle direto (lookup tables): `cols[N]` para rastreamento de colunas, `diag1[2N - 1]` e `diag2[2N - 1]` para as diagonais principal e secundária. Essa alocação permite a validação de ataques e a execução do critério de viabilidade (bound) em tempo constante $O(1)$.

- **Bitmask:** elimina a alocação de objetos e vetores na heap, utilizando três variáveis inteiras primitivas (rowMask, ld, rd) como máscaras de bits para representar as restrições de colunas e diagonais. As operações de atualização e consulta de posições válidas são realizadas por operadores lógicos e de deslocamento bit a bit em nível de registrador.

A partir dessas descrições, é possível estabelecer uma comparação com o que foi apresentado no referencial teórico. Enquanto as literaturas consultadas (GEEKSFORGEKS, 2026) e (GEEKSFORGEKS, 2025), respectivamente, descrevem implementações do Backtracking e Branch and Bound baseadas em uma matriz $n \times n$ para representações do tabuleiro – resultando em complexidade espacial $O(n^2)$ –, as implementações realizadas neste trabalho utilizam estruturas mais enxutas (vetor unidimensional e vetores booleanos, respectivamente), reduzindo a complexidade espacial auxiliar para $O(n)$ em ambos os casos. Por outro lado, a variação do Backtracking tradicional (Backtracking com Bit-masking) permanece com sua complexidade espacial $O(n)$ justamente por não utilizar arrays ou matrizes, mas 3 variáveis inteiras primitivas (rowMask, ld, rd).

3.3. Análise de Corretude e Desempenho por algoritmo

A corretude de um algoritmo refere-se à garantia de que, para qualquer entrada válida, o algoritmo produz a saída esperada, respeitando as condições impostas pelo problema. No caso dos algoritmos analisados, essa garantia é demonstrada por meio de uma prova de invariante de laço, que verifica se uma determinada propriedade é mantida verdadeira antes, durante e após cada execução do laço ou chamada recursiva. Complementarmente, é apresentada a análise de desempenho de cada implementação, na qual são derivadas as complexidades de tempo e de espaço a partir das estruturas de dados e do comportamento recursivo efetivamente empregados no código.

3.3.1. Backtracking

3.3.1.1. Análise de Corretude

Invariante de Laço

A cada chamada de `solveNQueens(board, row, n)`, o vetor `board[0...row-1]` armazena uma solução parcial válida de `row` rainhas nas primeiras `row` linhas, sem que duas rainhas compartilhem a mesma coluna, mesma diagonal principal ou a mesma diagonal secundária. Essa validade é garantida por `isSafe(board, row, col)`, que compara a posição candidata com todas as rainhas já posicionadas nas linhas anteriores, checando conflito de coluna (`board[i] == col`) e de diagonal (`Math.abs(board[i] - col) == Math.abs(i - row)`).

Inicialização

Antes que ocorra a primeira chamada do algoritmo, o estado é invocado com `solveNQueens(board, 0, n)`. Neste momento inicial, nenhuma rainha foi posicionada (`row = 0`), portanto a solução parcial vazia satisfaz trivialmente a invariante — não há conflitos porque não há rainhas alocadas. Assim, a invariante é verdadeira no início do algoritmo.

Manutenção

Para cada chamada do algoritmo recursivo no nível `row`, o laço `for` percorre as `N` colunas candidatas. Para cada coluna, `isSafe` verifica em $O(row)$ se a posição conflita com alguma rainha das linhas anteriores. Se a posição for segura, a rainha é registrada em `board[row] = col` e a recursão avança para `solveNQueens(board, row + 1, n)`, preservando a invariante para `row + 1`. Se a posição não for segura, a poda é contabilizada (`podas++`) e nenhuma rainha inválida é registrada, mantendo o vetor `board` consistente com a invariante. Diferente do Branch and Bound, não existem arrays booleanos auxiliares para reverter após o retorno da recursão — cada posição de `board` é apenas sobrescrita quando uma nova tentativa válida ocorre na mesma linha. Assim, a invariante permanece verdadeira a cada passo do algoritmo.

Término

O caso base é alcançado quando `row == n`, significando que as `N` rainhas foram posicionadas sem conflitos entre si. Pela invariante, o estado alcançado corresponde a uma solução válida e exata do problema, momento em que o algoritmo contabiliza o sucesso (`solutions++`) e salva a primeira solução encontrada (`firstSolution`). O algoritmo necessariamente termina porque, a cada chamada recursiva, `row` aumenta estritamente em 1 e é limitado superiormente por `n`, e o laço `for` em cada nível itera exatamente `n` vezes, garantindo que a árvore de busca é finita.

3.3.1.2. Análise de Desempenho

O desempenho do Backtracking clássico está diretamente ligado à exploração do espaço de possibilidades para o posicionamento das N rainhas. A cada linha, o algoritmo tenta posicionar uma rainha em cada uma das N colunas, e a validação de segurança de cada tentativa (`isSafe`) percorre todas as n rainhas já posicionadas nas linhas anteriores, custando $O(n)$ no pior caso. Como essa checagem $O(n)$ é executada para cada uma das N colunas testadas em cada nó da árvore de busca, o custo de trabalho por nó chega a $O(n^2)$.

Quanto ao tamanho da árvore de busca, no pior caso o algoritmo explora um número de nós limitado por $O(n!)$, já que a restrição de linha e coluna reduz progressivamente o número de colunas candidatas a cada nível, aproximando o crescimento do espaço de busca ao de uma permutação das N colunas entre as N linhas. Combinando o custo de $O(n^2)$ por nó com o limite de $O(n!)$ nós, obtém-se um limite superior de $O(n^2 \cdot n!)$ para o tempo de execução no pior caso.

É importante destacar que toda a poda realizada pelo algoritmo é reativa: o corte de um ramo só ocorre no momento em que uma coluna específica é testada e rejeitada por `isSafe` (`podas++`), nunca antes disso. Não há nenhuma verificação de viabilidade antecipada sobre as linhas seguintes — o algoritmo sempre desce um nível na recursão para, só então, testar cada coluna uma a uma. Essa característica é intrínseca à estratégia de Backtracking clássico e decorre diretamente de sua definição: a busca por retrocesso avança enquanto for possível e só recua quando encontra um beco sem saída.

Em relação ao consumo de memória, o algoritmo utiliza apenas um array `board` de tamanho N para representar a solução parcial corrente, sem nenhuma estrutura auxiliar adicional. A profundidade máxima da pilha de recursão é N , uma vez que cada chamada recursiva corresponde ao posicionamento de uma rainha em uma nova linha. Como o algoritmo não armazena todas as soluções nem os nós já visitados da árvore de busca, o consumo de memória permanece linear em relação a N , resultando em complexidade espacial $O(n)$.

3.3.2. Branch and Bound

3.3.2.1. Análise de Corretude

Invariante de Laço

A cada chamada de `solveNQueens(board, row, n)`, o vetor `board[0...row-1]` armazena uma solução parcial válida de *row* rainhas nas primeiras *row* linhas, sem que duas rainhas compartilhem a mesma coluna, mesma diagonal principal ou a mesma diagonal secundária. Os arrays booleanos `cols`, `diag1` e `diag2` mapeiam com exatidão as colunas (*col*), diagonais principais (*row + col*) e diagonais secundárias (*row - col + n - 1*) atualmente sob ataque pelas rainhas já alocadas, garantindo que as restrições do problema sejam respeitadas.

Inicialização

Antes que ocorra a primeira chamada do algoritmo, o estado é invocado com `solveNQueens(board, 0, n)`. Neste momento inicial, todos os arrays booleanos valem `false`, o que é coerente e correto já que nenhuma rainha foi posicionada no tabuleiro, indicando que nenhuma casa está sob ataque. Assim, a invariante é verdadeira no início do algoritmo.

Manutenção

Para cada chamada do algoritmo recursivo no nível *row*, a validade da solução parcial é mantida por meio das tabelas de lookup booleanas. As posições seguras da linha atual são identificadas em $O(1)$ pela conjunção `!cols[col] && !diag1[row+col] && !diag2[row-col+n-1]`. Ao encontrar uma coluna válida, a rainha é registrada em `board[row] = col` e as marcações de ataque são ativadas. Antes de descer para a próxima linha, a função `existeColunaViavel(row+1, n)` atua como *bound*, verificando se a linha seguinte ainda possui ao menos uma posição livre; caso não possua, o ramo inteiro é podado sem gerar chamadas recursivas para a subárvore inviável. Após o retorno da recursão, as marcações são restauradas para `false`, reestabelecendo a invariante para a próxima iteração de coluna no mesmo nível. Assim, a invariante permanece verdadeira a cada passo do algoritmo.

Término

O caso base é alcançado quando `row == n`, significando que as *N* rainhas foram posicionadas sem conflitos. Pela invariante, o estado alcançado corresponde a uma solução válida e exata do problema, momento em que o algoritmo contabiliza o sucesso (`solutions++`) e retorna. O algoritmo necessariamente termina porque, a cada chamada recursiva, *row* aumenta estritamente em 1 e é limitado superiormente por *n*,

e o laço for em cada nível itera exatamente n vezes, garantindo que a árvore de busca é finita.

3.3.2.2. Análise de Desempenho

O desempenho do algoritmo Branch and Bound está relacionado à exploração do espaço de possibilidades para posicionamento das N rainhas. No pior caso, a árvore de busca pode possuir uma quantidade fatorial de possibilidades, sendo possível considerar um limite superior de $O(n!)$ nós explorados.

Em cada nó, o laço for percorre as N colunas do tabuleiro. A verificação de validade de cada posição é realizada em $O(1)$, utilizando os arrays booleanos `cols`, `diag1` e `diag2`. Além disso, para cada posição válida, o algoritmo pode executar a função `existeColunaViavel`, que percorre até N colunas da próxima linha, apresentando custo $O(n)$. Dessa forma, considerando o pior caso, o trabalho realizado em cada nó pode chegar a $O(n^2)$. Assim, utilizando o limite superior de $O(n!)$ nós, obtém-se um limite superior de $O(n^2 \cdot n!)$ para o tempo de execução.

Entretanto, esse limite representa um pior caso e não significa que todas essas possibilidades sejam efetivamente exploradas. A estratégia Branch and Bound realiza uma poda antecipada por meio da função `existeColunaViavel`. Quando nenhuma posição válida é encontrada para a próxima linha, o algoritmo encerra aquele ramo imediatamente, evitando a criação e exploração de toda a subárvore correspondente. Portanto, na prática, essa estratégia reduz o espaço de busca e a quantidade de chamadas recursivas, embora o volume exato de podas dependa do valor de N e das configurações encontradas durante a execução.

Em relação ao consumo de memória, o algoritmo utiliza arrays de tamanho linear: `board` possui tamanho N , `cols` possui tamanho N e `diag1` e `diag2` possuem tamanho $2N - 1$. Além disso, a profundidade máxima da recursão é N . Como o algoritmo não armazena todas as soluções ou todos os nós da árvore de busca, o consumo de memória permanece linear em relação a N , resultando em complexidade espacial $O(n)$.

3.3.3. Backtracking com bit-masking

3.3.3.1. Análise de Corretude

Invariante de Laço

A cada chamada do método `solveNQueens`, as variáveis inteiras `rowMask`, `ld` e `rd` representam uma solução parcial válida do tabuleiro. Os bits ativados nessas variáveis mapeiam com exatidão as colunas, as diagonais esquerdas e as diagonais direitas que já estão sob ataque pelas rainhas alocadas nas linhas anteriores, garantindo que restrições geométricas do problema sejam respeitadas.

Inicialização

Antes que ocorra a primeira chamada do algoritmo, o estado é invocado com `solveNQueens(0, 0, 0)`. Neste momento inicial, as máscaras valem zero, o que é coerente e correto já que nenhuma rainha foi posicionada no tabuleiro, indicando que nenhuma casa está sob ataque no momento. Assim, a invariante é verdadeira no início do algoritmo.

Manutenção

Para cada chamada do algoritmo recursivo, a validade da solução parcial é mantida por meio de lógica booleana diretamente nos registradores da CPU. As posições seguras da linha atual são extraídas de forma simultânea pela operação `pos = done & (~(rowMask | ld | rd))`. Ao iterar sobre os bits livres (representando colunas seguras), a chamada para a próxima linha atualiza o estado de forma determinística: `rowMask | bit` adiciona o ataque na mesma coluna; `(ld | bit) << 1` projeta o ataque da diagonal esquerda para a linha abaixo; e `(rd | bit) >> 1` projeta o ataque da diagonal direita. Caso não haja bits livres (`pos == 0`), o laço não executa (poda), descartando caminhos inviáveis. Assim, a invariante permanece verdadeira a cada passo do algoritmo.

Término

No fim de todas as chamadas recursivas iterativas, o caso base é alcançado quando a máscara `rowMask` tem todos os seus N bits preenchidos (`rowMask == done`), significando que as N rainhas foram consideradas e posicionadas. Pela invariante, temos que o estado alcançado corresponde a uma solução válida e exata do problema, momento em que o algoritmo contabiliza o sucesso (`solutions++`) e retorna.

3.3.3.2. Análise de Desempenho

O desempenho do algoritmo Bitmask se baseia na exploração da árvore completa do espaço de soluções possíveis. Como a natureza do problema das N -

Queens exige a busca e enumeração de todas as configurações válidas, a complexidade temporal segue um limite superior exponencial/fatorial, comportando-se no pior caso como $O(n!)$.

Além disso, por não ser baseado em matrizes, o algoritmo reduz drasticamente a constante de custo de cada operação. Isso se deve à verificação de todas as restrições de uma linha em tempo constante $O(1)$, utilizando operadores bit a bit nativos da arquitetura (AND, OR, NOT e Shifts). Outro aspecto notável é o comportamento do laço iterativo while ($pos > 0$), o qual não percorre o tabuleiro inteiro, iterando apenas sobre o número exato de casas seguras, extraíndo o bit menos significativo em cada passo pela instrução $bit = pos \& -pos$. Portanto, devido a essas estratégias de eficiência e tratamento em baixo nível, há uma redução drástica nas iterações e ramificações inúteis na prática.

Em relação ao consumo de memória do algoritmo, é apresentado um comportamento eficaz, já que é utilizado um espaço correspondente ao tamanho da entrada. O Bitmask elimina completamente o instanciamento dinâmico de arrays ou matrizes na memória (*Heap*). As estruturas de apoio resumem-se exclusivamente a três tipos primitivos int (32 bits cada). Dado que o algoritmo não guarda todas as soluções viáveis na memória, mas sim propaga o estado via passagem de valor, o espaço consumido está diretamente ligado à profundidade da recursão, que possui o limite máximo de N chamadas. Portanto, o custo de espaço adicional é de $O(1)$ por nó na árvore, mantendo a complexidade espacial global estritamente linear em $O(n)$.

3.4. Critérios Experimentais e Coleta de Métricas

A efeito de comparação, realizou-se uma análise empírica de desempenho entre os algoritmos Backtracking clássico, Branch and Bound e Bitmask para a resolução exata do problema das N-Queens (N-rainhas). A investigação teve como objetivo quantificar a complexidade temporal, o consumo de memória e a eficácia na poda do espaço de busca sob diferentes dimensões do tabuleiro (N). Essa investigação possui um caráter estritamente acadêmico e visou averiguar a eficiência prática de cada estratégia frente ao crescimento do espaço de busca.

Para a medição do tempo de execução em milissegundos (ms), utilizou-se no estudo o método nativo `System.nanoTime()` da linguagem Java. O consumo de memória alocada (KB) foi monitorado por meio da classe `Runtime`, capturando a

diferença entre a memória total e a memória livre (`totalMemory()` – `freeMemory()`), precedida por uma chamada explícita ao coletor de lixo (`System.gc()`) antes de cada execução a fim de minimizar ruídos residuais da JVM. Adicionalmente, implementou-se a contagem estrita de duas variáveis de controle algorítmico: a quantidade total de chamadas recursivas (nós explorados) e o número de podas efetuadas durante a busca em profundidade.

Os experimentos foram delineados sob três frentes metodológicas principais:

- 1. Instâncias de Teste:** o intervalo de análise estendeu-se de $N = 4$ até $N = 14$. Os casos triviais ($N = 0, 1, 2, 3$) foram desconsiderados na análise comparativa, visto que $N = 1$ possui solução imediata unitária e $N = 0, 2$ ou 3 não admitem soluções válidas.
- 2. Benchmark e Repetibilidade:** para mitigar a variabilidade térmica e o escalonamento de processos dos sistemas operacionais, cada instância de N foi submetida a uma bateria de 50 repetições consecutivas em cada uma das quatro máquinas de teste. A partir dessas rodadas, calcularam-se as médias aritméticas de tempo e memória, as quais foram persistidas automaticamente em arquivos no formato .csv.
- 3. Teste de Viabilidade (Ponto de Ruptura):** conduziu-se um teste de estresse temporal para identificar a dimensão N limite na qual cada estratégia se torna computacionalmente inviável para uso prático. Para este teste, estabeleceu-se como critério uniforme de parada o limiar de corte de 1 minutos (60 segundos) de execução contínua; a primeira dimensão de N que ultrapassou esse intervalo delimitou o ponto de ruptura do respectivo algoritmo.

Consolidados os parâmetros de teste e os procedimentos de coleta, a seção subsequente apresenta os resultados obtidos com os experimentos, acompanhados, na posterior, das análises comparativas de desempenho e discussões acerca do comportamento dos algoritmos.

N	Abordagem	Média de 50 Testes				
		Tempo (ms)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
4	Backtracking	0,02	2,54	17	44	2
	B&B	0,04	7,46	13	32	2
	Bitmask	0	0	17	44	2
9	Backtracking	1,58	2,62	8394	63985	352
	B&B	1,04	2,62	5810	43313	352
	Bitmask	0,11	0	8394	63985	352
14	Backtracking	7.625,67	2,63	27358553	350542846	365596
	B&B	4106,18	2,63	17693457	224896598	365596
	Bitmask	341,6	0	27358553	350542846	365596

Fonte: Elaborado pelos autores (2026).

Tabela 6 - Desempenho dos Algoritmos Backtracking, Branch and Bound e Bitmask no problema das N-Rainhas (Máquina 3)

COMPARAÇÃO POR ALGORITMO (Máquina 3)						
N	Abordagem	Média de 50 Testes				
		Tempo (ms)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
4	Backtracking	0,01	32,06	17	44	2
	B&B	0,02	33,15	13	32	2
	Bitmask	0	0	17	44	2
9	Backtracking	1,27	32,73	8394	63985	352
	B&B	0,8	32,73	5810	43313	352
	Bitmask	0,09	0	8394	63985	352
14	Backtracking	4495,31	32,75	27358553	350542846	365596

	B&B	2584,67	32,75	17693457	224896598	365596
	Bitmask	206,52	0	27358553	350542846	365596

Fonte: Elaborado pelos autores (2026).

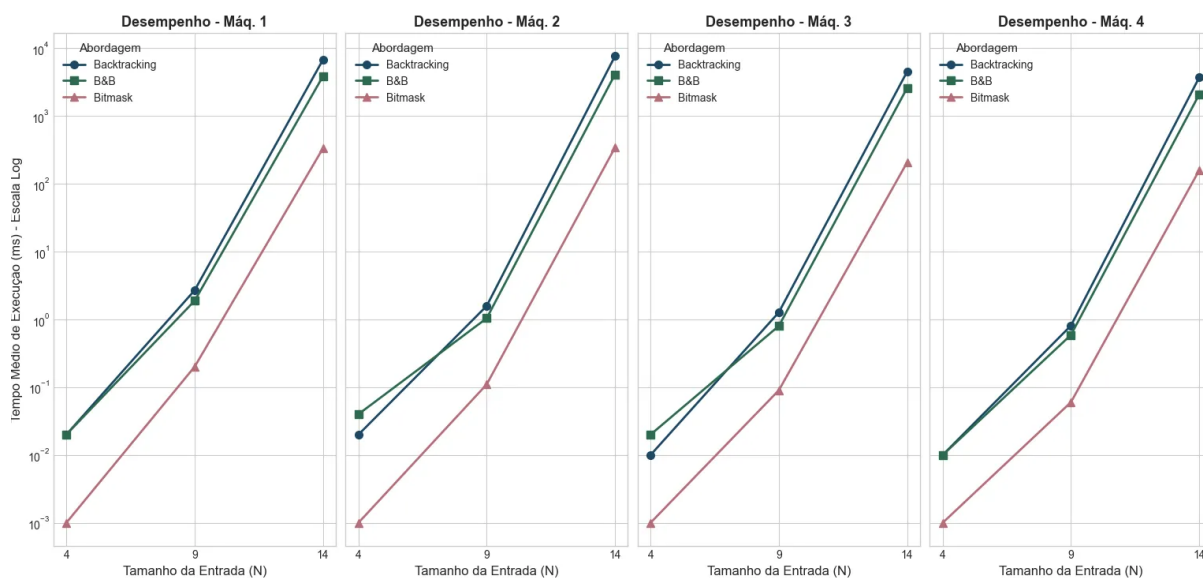
Tabela 7 - Desempenho dos Algoritmos Backtracking, Branch and Bound e Bitmask no problema das N-Rainhas (Máquina 4)

COMPARAÇÃO POR ALGORITMO (Máquina 4)						
N	Abordagem	Média de 50 Testes				
		Tempo (ms)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
4	Backtracking	0,01	2,54	17	44	2
	B&B	0,01	6,64	13	32	2
	Bitmask	0	0	17	44	2
9	Backtracking	0,81	2,62	8394	63985	352
	B&B	0,59	2,62	5810	43313	352
	Bitmask	0,06	0	8394	63985	352
14	Backtracking	3701,89	2,63	27358553	350542846	365596
	B&B	2073,29	2,63	17693457	224896598	365596
	Bitmask	157,79	0	27358553	350542846	365596

Fonte: Elaborado pelos autores (2026).

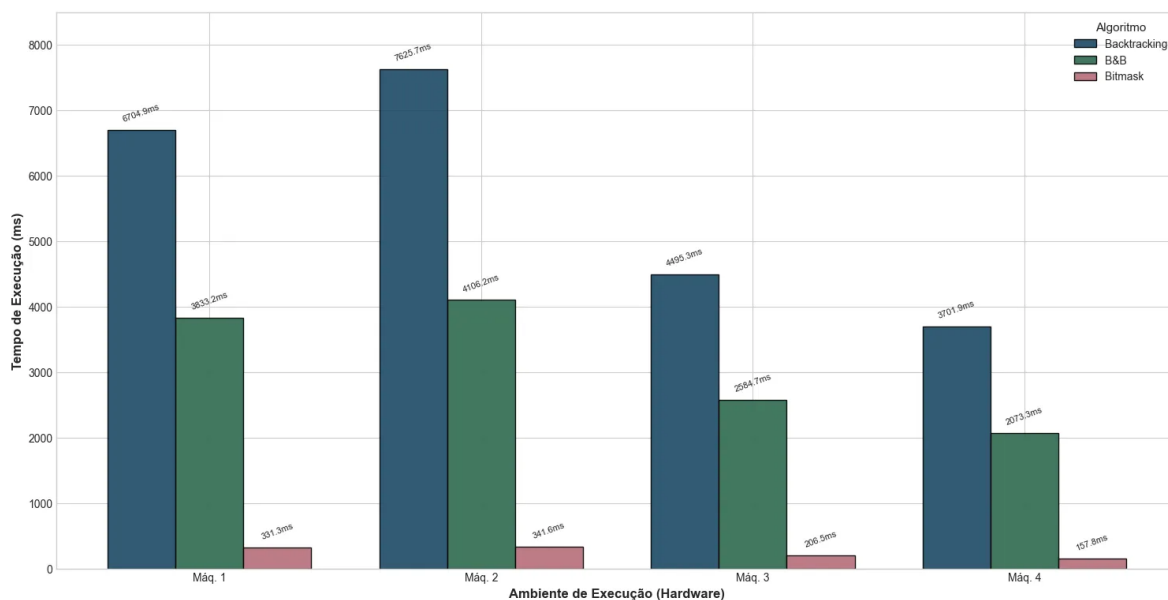
As tabelas 4 a 7 consolidam as métricas de desempenho obtidas na execução dos algoritmos para o problema das N-Rainhas em quatro ambientes computacionais distintos (máquinas 1 a 4). Para assegurar uma síntese representativa do comportamento assintótica sem sobrecarregar a exibição dos dados, aplicou-se o particionamento em classes de equivalência sobre o intervalo de testes ($N = 4$ a $N = 14$), destacando três cenários estratégicos: $N = 4$ (limite inferior / início), $N = 9$ (ponto intermediário) e $N = 14$ (limite superior / fim). Os gráficos correspondentes ilustram a evolução dessas métricas e fornecem o suporte quantitativo para as análises e comparações aprofundadas na seção de Discussão.

Figura 3 - Tempo médio de execução em escala logarítmica em função do tamanho da entrada (N) por máquina



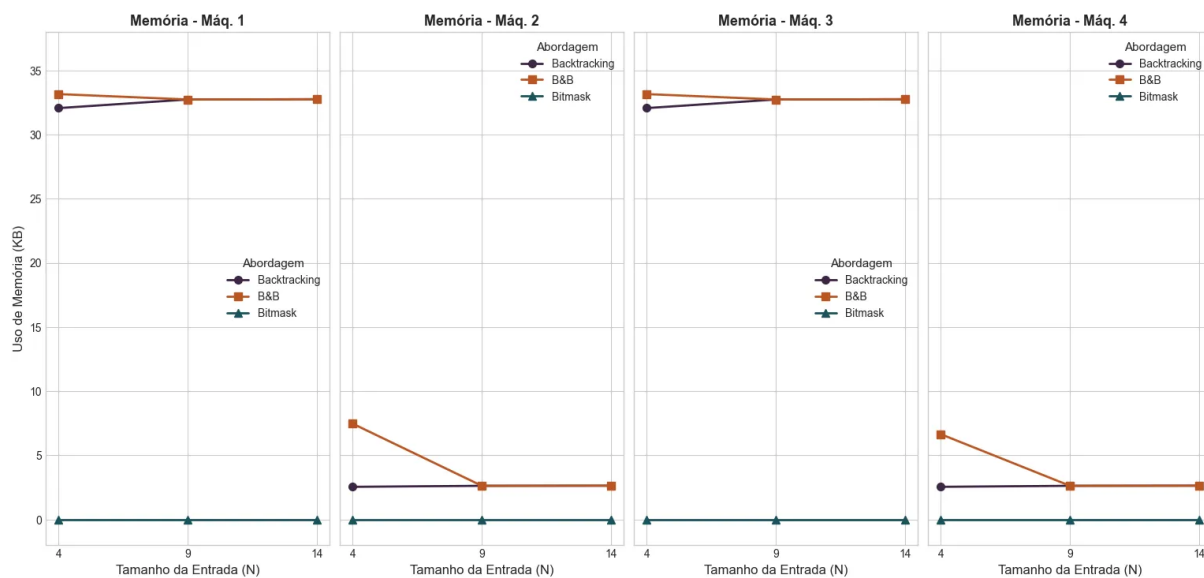
Fonte: Elaborado pelos autores (2026).

Figura 4 - Comparativo de tempo de execução entre os algoritmos para o caso crítico (N = 14) em diferentes ambientes de hardware



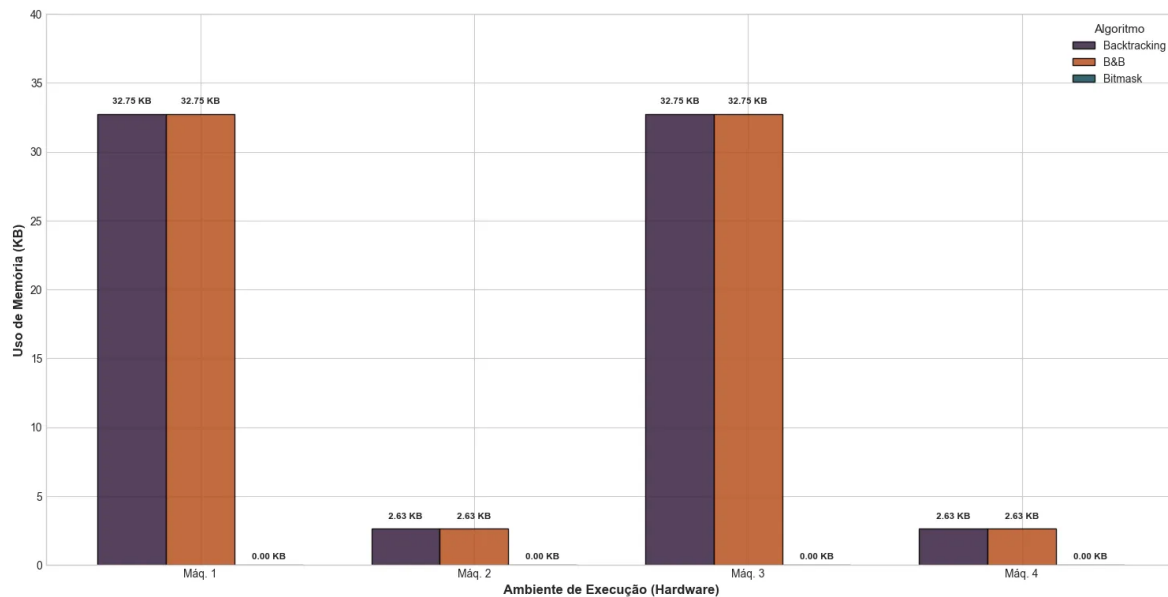
Fonte: Elaborado pelos autores (2026).

Figura 5 - Consumo médio de memória dos algoritmos em função do tamanho da entrada (N)



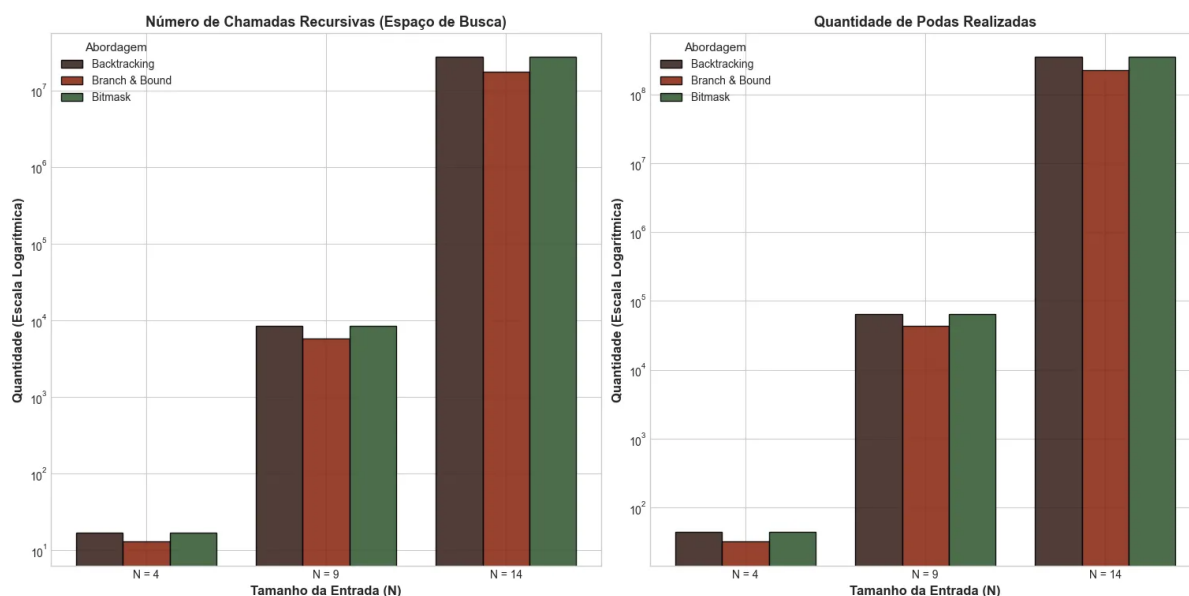
Fonte: Elaborado pelos autores (2026).

Figura 6 - Comparativo de consumo de memória dos algoritmos entre os ambientes de execução



Fonte: Elaborado pelos autores (2026).

Figura 7 - Número de chamadas recursivas e quantidade de podas por abordagem no problema das N-Rainhas



Fonte: Elaborado pelos autores (2026).

Os gráficos apresentados sintetizam as variáveis empíricas que fundamentarão a seção de Discussão. As Figuras 3 e 4 analisam o desempenho temporal das abordagens – com destaque para o caso crítico em $N = 14$ na Figura 4 –, enquanto as Figuras 5 e 6 avaliam o consumo de memória entre as diferentes máquinas. Por fim, a Figura 7 detalha a exploração da árvore de recursão por meio das podas e chamadas realizadas. A contagem de soluções encontradas foi omitida dos gráficos, visto que se trata de uma métrica de corretude algorítmica, cujos valores foram idênticos em todas as execuções.

4.2. Teste de Viabilidade (Ponto de Ruptura)

Tabela 8 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 1)

INVIABILIDADE DE ENTRADA (Máquina 1)						
Limiar de 1 Minuto						
Abordagem	N	Tempo (s)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
Backtracking	16	278,69	32,76	1141190303	16881494354	14772512
B&B	16	127,19	32,76	740447439	10870351394	14772512

Bitmask	17	65,19	0	8017021932	126643494145	95815104
---------	----	-------	---	------------	--------------	----------

Fonte: Elaborado pelos autores (2026).

Tabela 9 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 2)

INVIABILIDADE DE ENTRADA (Máquina 2)						
Limiar de 1 Minuto						
Abordagem	N	Tempo (s)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
Backtracking	16	338,47	2,64	1141190303	16881494354	14772512
B&B	16	181,2	2,64	740447439	10870351394	14772512
Bitmask	17	101,88	0	8017021932	126643494145	95815104

Fonte: Elaborado pelos autores (2026).

Tabela 10 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 3)

INVIABILIDADE DE ENTRADA (Máquina 3)						
Limiar de 1 Minuto						
Abordagem	N	Tempo (s)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
Backtracking	16	205,37	32,76	1141190303	16881494354	14772512
B&B	16	109,13	32,76	740447439	10870351394	14772512
Bitmask	17	64,49	0	8017021932	126643494145	95815104

Fonte: Elaborado pelos autores (2026).

Tabela 11 - Limite de viabilidade prática de execução sob limiar de 1 minuto (Máquina 4)

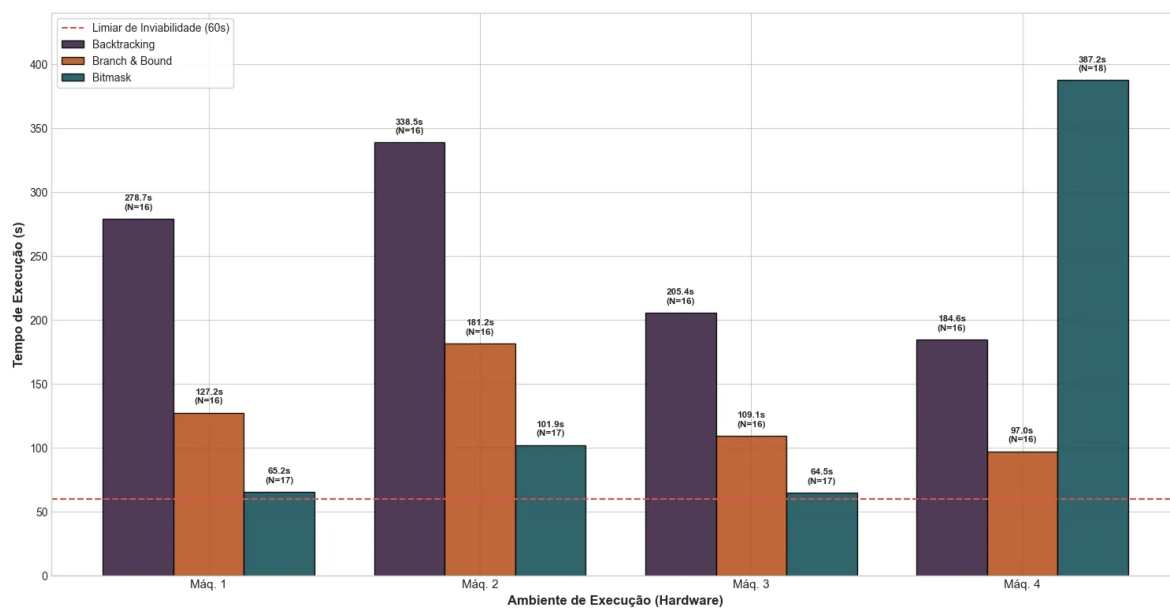
INVIABILIDADE DE ENTRADA (Máquina 4)						
Limiar de 1 Minuto						
Abordagem	N	Tempo (s)	Memória (KB)	Nº de Chamadas Recursivas	Podas	Soluções
Backtracking	16	184,55	2,64	1141190303	16881494354	14772512

B&B	16	96,98	2,64	740447439	108703513 94	14772512
Bitmask	18	387,22	0	5936584449 1	997229725 116	66609062 4

Fonte: Elaborado pelos autores (2026).

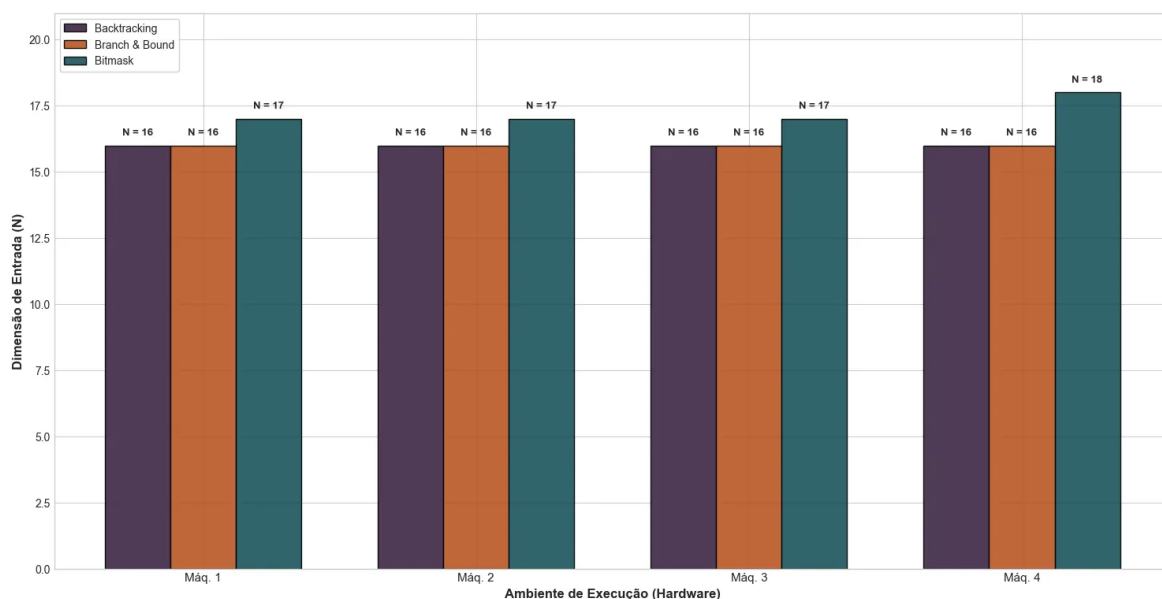
As Tabelas 8 a 11 consolidam as métricas associadas ao limiar de inviabilidade prática de execução dos algoritmos para o problema das N-Rainhas em quatro ambientes computacionais distintos (máquinas 1 a 4). Para essa síntese, considerou-se o maior valor de N processado sob a restrição de tempo pré-estabelecida (limiar de 1 minuto). As representações gráficas complementares ilustram o comportamento dessas métricas, fornecendo o embasamento quantitativo para as análises comparativas aprofundadas na seção de Discussão.

Figura 8 - Tempo de execução no ponto de ruptura por abordagem sob o limiar de 1 minuto em diferentes ambientes de hardware



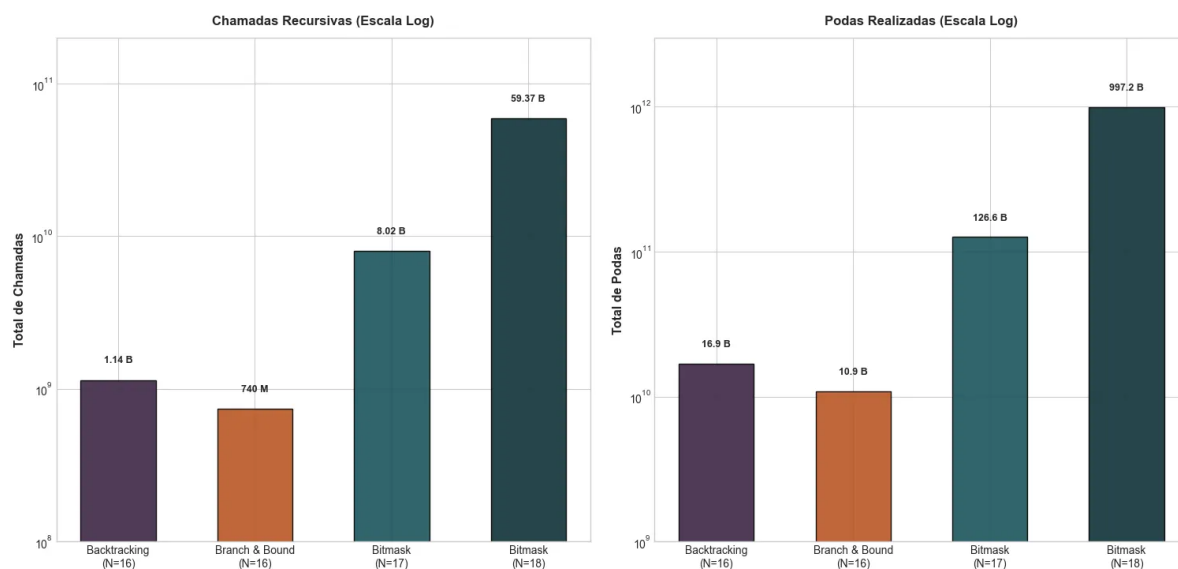
Fonte: Elaborado pelos autores (2026).

Figura 9 - Dimensão máxima de entrada (N) processada por abordagem até o esgotamento do limiar temporal



Fonte: Elaborado pelos autores (2026).

Figura 10 - Comportamento do espaço de busca (chamadas recursivas e podas) nos cenários limites de processamento (N = 16, 17, 18)



Fonte: Elaborado pelos autores (2026).

Os gráficos apresentados sintetizam as variáveis empíricas que fundamentam as análises da seção de Discussão. A Figura 8 apresenta o tempo de execução no ponto de ruptura em cada máquina, enquanto a Figura 9 compara a dimensão máxima

de entrada processada. Por fim, a Figura 10 ilustra o comportamento do espaço de busca em termos de chamadas recursivas e podas. Por outro lado, as métricas de consumo de memória e contagem de soluções dispensaram representação gráfica complementar devido sua baixa magnitude e invariância determinística, mantendo-se restritas às tabelas.

5. DISCUSSÃO

Conforme evidenciado nas Tabelas 4 a 7 e nas Figuras 3 e 4, o algoritmo do Backtracking com Bitmask destacou-se substancialmente em termos de tempo de execução frente às demais abordagens. Esse comportamento consolida a análise teórica: ao contrário das versões baseadas em arrays, a abordagem via máscara de bits reduz drasticamente o fator constante de custo associado a cada operação. A validação de ataques nas colunas e diagonais é resolvida em tempo $O(1)$ por meio de Operadores bit a bit nativos da arquitetura (AND, OR, NOT e Shifts). Além disso, a instrução `bit = pos & -pos` permite iterar estritamente sobre as posições seguras sem a necessidade de varrer células inválidas da linha.

Ademais, é válido ressaltar que essa otimização do algoritmo não reduz a árvore de estados – visto que o Bitmask e o Backtracking clássico exploram exatamente a mesma quantidade de chamadas recursivas nas 4 máquinas (27.358.553 nós em $N = 14$) –, mas acelera o processo por nó, tornando o Bitmask de 20 a 23 vezes mais rápido que o Backtracking clássico e cerca de 11 a 13 vezes mais rápido que o Branch and Bound no cenário crítico, conforme explicitado na Figura 4. Dessa forma, embora compartilhem da mesma complexidade assintótica de pior caso $O(n!)$, o impacto prático da redução do fator constante e da eliminação de overhead na memória resulta em uma eficiência temporal superior.

Paralelamente a esse cenário, a Figura 3 evidencia de forma visível a transição de desempenho entre o Backtracking padrão e o Branch and Bound. Para entradas muito pequenas ($N = 4$), o Branch and Bound apresenta desempenho equivalente ou ligeiramente inferior ao Backtracking tradicional nas quatro máquinas avaliadas. Esse comportamento ocorre devido ao custo computacional extra (overhead) gerado pela alocação das tabelas de lookup (cols, diag1, diag2) e pelas sucessivas invocações da função de validação preditiva `existeColunaViavel`, cujo custo $O(n)$ não compensa em árvores de busca rasas. Contudo, à medida que a dimensão do tabuleiro cresce em direção a $N = 9$ e $N = 14$, a poda antecipada de ramos torna-se determinante: em $N = 14$, por exemplo, o Branch and Bound explora apenas 17.693.457 chamadas recursivas contra 27.358.553 do Backtracking clássico, eliminando mais de 9,6 milhões de nós da árvore de estados. Essa redução estrutural no espaço de busca traduz-se em ganhos temporais expressivos, consolidando o Branch and Bound mais eficientes em cenários mais críticos.

No que tange ao consumo de recursos de memória, as Figura 5 e 6 ilustram o perfil de alocação das três abordagens nos diferentes ambientes computacionais. Observa-se que, a partir das Tabelas 4 a 7, o Backtracking clássico e o Branch and Bound mantiveram perfis de consumo muito próximos e contidos – na ordem de 2,6 KB nas máquinas 2 e 4, e cerca de 32,7 KB nas máquinas 1 e 3 –, o que reafirma a modelagem adotada na Metodologia, onde o uso de vetores unidimensionais garantiu uma complexidade espacial linear $O(n)$ para ambas as estratégias. Por sua vez, o Backtracking com Bitmask registrou consumo nulo de alocação dinâmica (0 KB) em todas as instâncias e máquinas avaliadas. Esse resultado empírico reflete a eliminação completa de estruturas instanciadas na *Heap*, uma vez que a propagação das restrições ocorre exclusivamente por meio de variáveis primitivas manipuladas diretamente na pilha de recursão, preservando a complexidade $O(n)$ com a menor pegada de memória possível.

Em complemento às menções anteriores acerca do espaço de busca, a Figura 7 detalha minuciosamente o comportamento algorítmico a partir do volume de chamadas recursivas e da quantidade de podas efetuadas nos três cenários de benchmark. Essa representação gráfica consolida a superioridade estrutural do Branch and Bound na contenção da árvore de estado: enquanto o Backtracking e o Bitmask executam exatamente a mesma quantidade de chamadas recursivas (27.358.553 nós em $N = 14$), o Branch and Bound reduz esse total para 17.693.457. Esse ganho decorre da avaliação preditiva da função de *bound*, que interrompe a descida na árvore antes da instanciamento de subárvores inviáveis, eliminando mais de 9,6 milhões de nós da busca em profundidade e minimizando substancialmente o número total de podas reativas necessárias.

Estabelecido o comparativo no intervalo regular do benchmark, as Tabelas 8 a 11 e as Figura 8 a 10 abordam a viabilidade prática e os pontos de ruptura dos algoritmos sob o limiar temporal de 1 minuto. Observa-se que, para o Backtracking clássico e o Branch and Bound, o limite de viabilidade foi uniforme em todos os dispositivos, esgotando o tempo limite invariavelmente de $N = 16$, com variações temporais decorrentes das diferentes capacidades de processamento (CPU) de cada máquina. Por sua vez, o Bitmask demonstrou uma capacidade de escala superior: nas Máquinas 1,2 e 3, o algoritmo alcançou $N = 17$ antes de ultrapassar o limiar; na Máquina 4, favorecido pelo processador de arquitetura mais recente (Core i5 de 13ª geração), o Bitmask suportou a execução até $N = 18$, processando mais de 5,9 bilhões

de nós e encontrando todas as 66.609.062 soluções válidas em 387,28 segundos. Esse comportamento consolida a extrema eficiência das operações em nível de bits em cenários de alta combinatória.

Por fim, a Figura 10 ilustra o aumento expressivo nas tentativas e caminhos testados nos cenários limites ($N = 16, 17$ e 18), detalhando o total de chamadas recursivas e podas realizadas por cada algoritmo. O gráfico deixa evidente o salto no processamento à medida que o tabuleiro aumenta: para $N = 16$, o Backtracking tradicional e o Branch and Bound alcançam, respectivamente, 1,14 bilhão e 740 milhões de chamadas recursivas, somando entre 10 e 16 bilhões de podas realizadas. Já nas entradas maiores de $N = 17$ e $N = 18$, processadas unicamente pelo Backtracking com Bitmask, as chamadas recursivas disparam para 8,02 bilhões e 59,37 bilhões, com as podas chegando à marca de quase 1 trilhão de tentativas descartadas (997, 2 bilhões em $N = 18$). Esses dados comprovam na prática que, mesmo que o Bitmask seja muito mais veloz em cada passo ou que o Branch and Bound corte caminhos inteiros, o crescimento exponencial do problema ainda impõe um limite prático real ao processamento em computadores convencionais.

6. CONSIDERAÇÕES FINAIS

O presente estudo cumpriu satisfatoriamente os objetivos propostos ao realizar uma investigação comparativa, tanto no plano teórico quanto no experimental, entre as abordagens de Backtracking clássico, Branch and Bound e Backtracking com Bitmask aplicadas à resolução exata do problema das N-Rainhas. As implementações desenvolvidas permitiram analisar a relação entre a complexidade assintótica formal e o comportamento real dos algoritmos sob diferentes arquiteturas de computadores, quantificando variáveis essenciais como tempo de processamento, consumo de memória, chamadas recursivas e podas de estados.

Os resultados obtidos evidenciaram de forma clara as particularidades e os compromissos de cada estratégia avaliada. O Branch and Bound destacou-se como a abordagem mais eficiente na contenção da árvore de busca, alcançando uma redução de aproximadamente 35% no número de chamadas recursivas por meio de sua função preditiva de *bound*, a qual evita a instanciação de subárvores inviáveis antes da descida recursiva. Por outro lado, o Backtracking com Bitmask consolidou-se como a técnica de maior desempenho temporal absoluto, processando as instâncias de 20 a 23 vezes mais rápido que o Backtracking clássico no cenário crítico. Essa vantagem decorre da manipulação direta de variáveis primitivas e operadores em nível de bits, o que também zerou o custo de alocação dinâmica de memória na *Heap* e viabilizou a resolução do problema até $N = 18$ no teste de estresse de 1 minuto. Já o Backtracking clássico serviu como linha de base experimental, evidenciando o custo do teste reativo individual e apresentando os maiores tempos de execução em todos os cenários testados.

Em suma, conclui-se que as otimizações estruturais adotadas — especialmente a representação do tabuleiro em vetores unidimensionais que garantiu complexidade espacial $O(n)$ para todas as abordagens — produzem impactos expressivos na redução dos fatores constantes de tempo e memória. Por outro lado, como o número de testes necessários cresce muito rápido a cada nova rainha, sempre haverá um limite prático para encontrar todas as respostas de forma sequencial.

7. DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA (IAGen)

Em conformidade com a Resolução UEPB/CONSEPE/031/2026, que estabelece a política institucional de uso responsável e transparente de Inteligência Artificial Generativa (IAGen) nas atividades acadêmicas, declara-se que ferramentas de IAGen foram utilizadas como recurso auxiliar de suporte técnico, refinamento textual e apoio à visualização de dados durante o desenvolvimento deste trabalho.

- **Ferramentas Utilizadas e Finalidades Específicas:**

- **Claude (Anthropic):** Apoio na análise crítica preliminar das complexidades assintóticas, apoio na estruturação e formatação das referências bibliográficas segundo as normas da ABNT, suporte na estruturação conceitual das provas de corretude (invariante de laço) e refinamento de trechos teóricos.
- **Gemini (Google):** montagem da estrutura inicial do projeto, apoio na revisão gramatical, aprimoramento da fluidez textual das seções metodológicas, suporte na geração dos scripts em Python (matplotlib / seaborn) para a plotagem dos gráficos de benchmark e viabilidade a partir dos dados brutos obtidos experimentalmente; e remodelação e recriação gráfica de ilustrações conceituais (diagramação do tabuleiro com nova paleta e tradução de termos para o português), garantindo a originalidade visual sem prejuízo da citação às fontes teóricas originais.

Conforme determinam os Arts.2º (§ 2) e 7º da referida normativa, todo o uso esteve estritamente sob o controle crítico, ético e intelectual dos autores. A modelagem dos algoritmos, o planejamento experimental, a coleta de métricas, a validação de todas as referências citadas e a redação final das análises constituem autoria intelectual exclusiva da equipe.

REFERÊNCIAS

ANDRADE, Gabriela Almeida de; GOUVEIA, Georis Samuel Martins; LACERDA, Horlan Silva de; MACIEL, Suelle Ferreira. **Análise comparativa de eficiência em algoritmos de busca e ordenação**. 43 f. Universidade Estadual da Paraíba, Campus I – Campina Grande, Paraíba, 2025. Disponível em: <<https://drive.google.com/drive/folders/1F-KGONXyjMdEmOnVVPsg-BO0gILl9KP4?usp=sharing>>. Acesso em: 09 ago. 2026.

SOUZA, Handerson Muller Ferro de. **Algoritmo eficiente para validação de soluções para o problema das N-Rainhas**. 43 f. Universidade Federal de Alagoas, Campus Arapiraca, Arapiraca, 2019. Disponível em: <<https://ud10.arapiraca.ufal.br/web/content?model=ud.biblioteca.anexo&field=arquivo&id=6181&download=true&filename=field=name>>. Acesso em: 09 ago. 2026.

OLIVEIRA, Gerson Pastre de. **Aspectos matemáticos do problema das n-rainhas e a construção do conhecimento por alunos de Ciência da Computação**. Educação Matemática Pesquisa, São Paulo, v. 26, n. 1, p. 642-667, 2024. Disponível em: <<http://dx.doi.org/10.23925/1983-3156.2024v26i1p642-667>>. Acesso em: 14 ago. 2026.

KARP, Richard M.; ZHANG, Yanjun. **Randomized parallel algorithms for backtrack search and branch-and-bound computation**. Journal of the ACM, v. 40, n. 3, p. 765-789, 1993. Disponível em: <<https://dl.acm.org/doi/10.1145/174130.174145>>. Acesso em: 14 ago. 2026.

MASSUIA, Giovanny. Backtracking. **Craft & Code Club**, 25 maio 2025. Disponível em: <<https://craftcodeclub.io/posts/dsa-backtracking>>. Acesso em: 14 ago. 2026.

RAI, Dilli Hang. **N-Queen Problem (BackTracking Algorithm)**. Medium, 12 set. 2024. Disponível em: <<https://medium.com/@dillihangrae/n-queen-problem-backtracking-algorithm-55ef786c3116>>. Acesso em: 15 ago. 2026.

SOUZA, Marcone Jamilson Freitas. **Otimização Combinatória**: notas de aula, 2009/2. Ouro Preto: Departamento de Computação, Instituto de Ciências Exatas e Biológicas, Universidade Federal de Ouro Preto, 2009. Disponível em: <<http://www.decom.ufop.br/prof/marcone>>. Acesso em: 15 ago. 2026.

GEEKSFORGEEKS. **N Queen Problem**. GeeksforGeeks, 2011. Atualizado em 4 abr. 2026. Disponível em: <<https://www.geeksforgeeks.org/dsa/n-queen-problem-backtracking-3/>>. Acesso em: 15 ago. 2026.

GEEKSFORGEEKS. **N Queen Problem Using Branch and Bound in C**. GeeksforGeeks, 2024. Atualizado em 23 jul. 2025. Disponível em: <<https://www.geeksforgeeks.org/c/n-queen-problem-using-branch-and-bound-in-c/>>. Acesso em: 15 ago. 2026.

UNIVERSIDADE ESTADUAL DA PARAÍBA (UEPB). Conselho de Ensino, Pesquisa e Extensão (CONSEPE). Resolução/UEPB/CONSEPE/031/2026: **Estabelece a política de uso responsável da Inteligência Artificial Generativa (IAGen) na Universidade Estadual da Paraíba (UEPB) e dá outras providências**. Campina Grande: UEPB, 21 jul. 2026. Disponível em: <<https://sites.uepb.edu.br/consepe/>>. Acesso em: 20 ago. 2026.